

SUMMARY

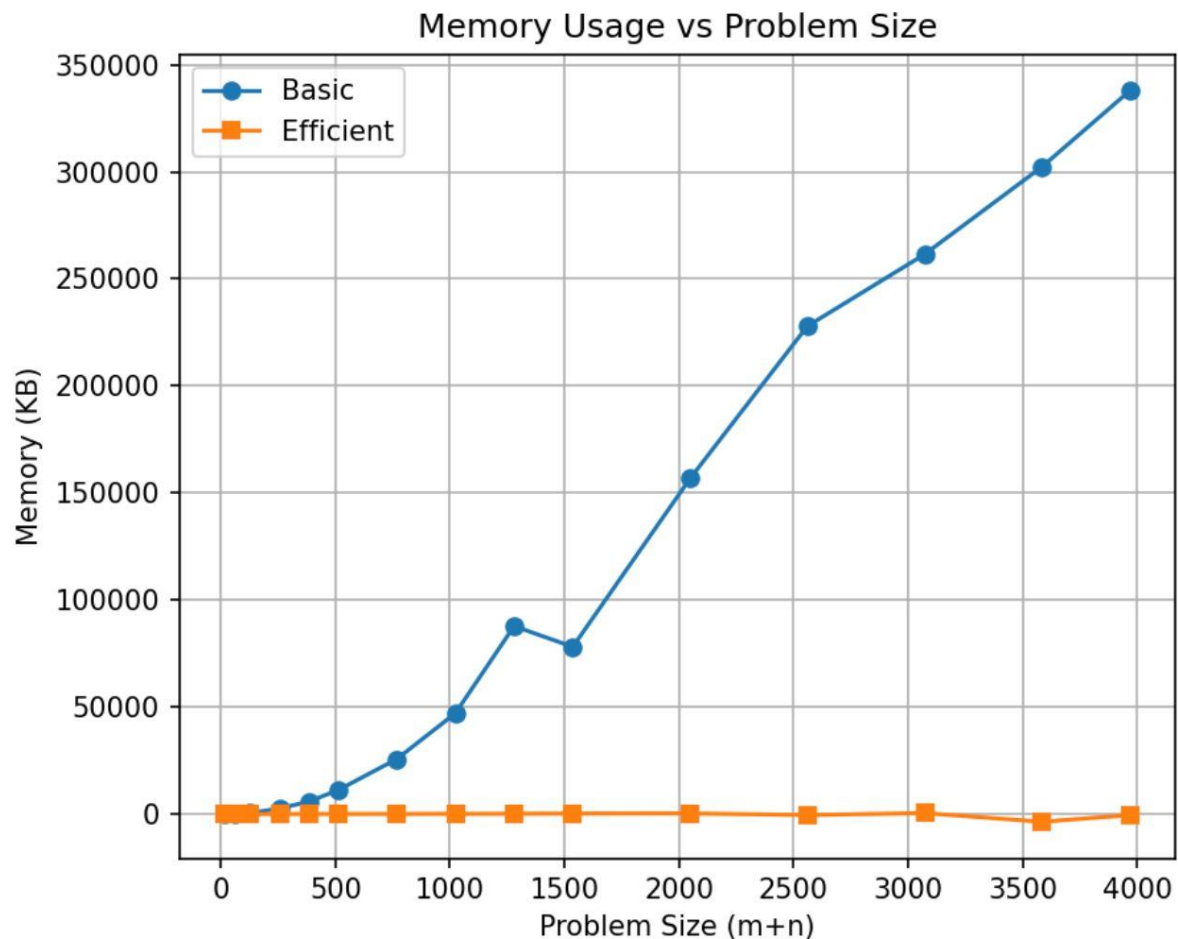
USC ID/s:

Datapoints

M+N	Time in MS (Basic)	Time in MS (Efficient)	Memory in KB (Basic)	Memory in KB (Efficient)
16	0.1418590545654297	0.1499652862548828	0	0
64	0.6346702575683594	0.80108642578125	96	0
128	3.083944320678711	3.628253936767578	512	0
256	9.050130844116211	11.045217514038086	2592	32
384	19.495010375976562	24.651050567626953	5856	64
512	34.49678421020508	44.04187202453613	11184	48
768	79.91504669189453	111.97423934936523	25712	64
1024	233.09588432312012	172.332763671875	47120	112
1280	258.61668586730957	265.549898147583	87792	144
1536	368.18838119506836	359.2700958251953	78112	256
2048	670.7348823547363	680.6051731109619	156720	288
2560	1142.6491737365723	1086.5740776062012	227904	-432
3072	1530.8749675750732	1433.161973953247	261472	432
3584	2192.6982402801514	2277.9459953308105	302320	-3584
3968	2734.6041202545166	2506.6280364990234	337920	-480

Insights

Graph1 – Memory vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

Basic: Polynomial

Efficient: Constant

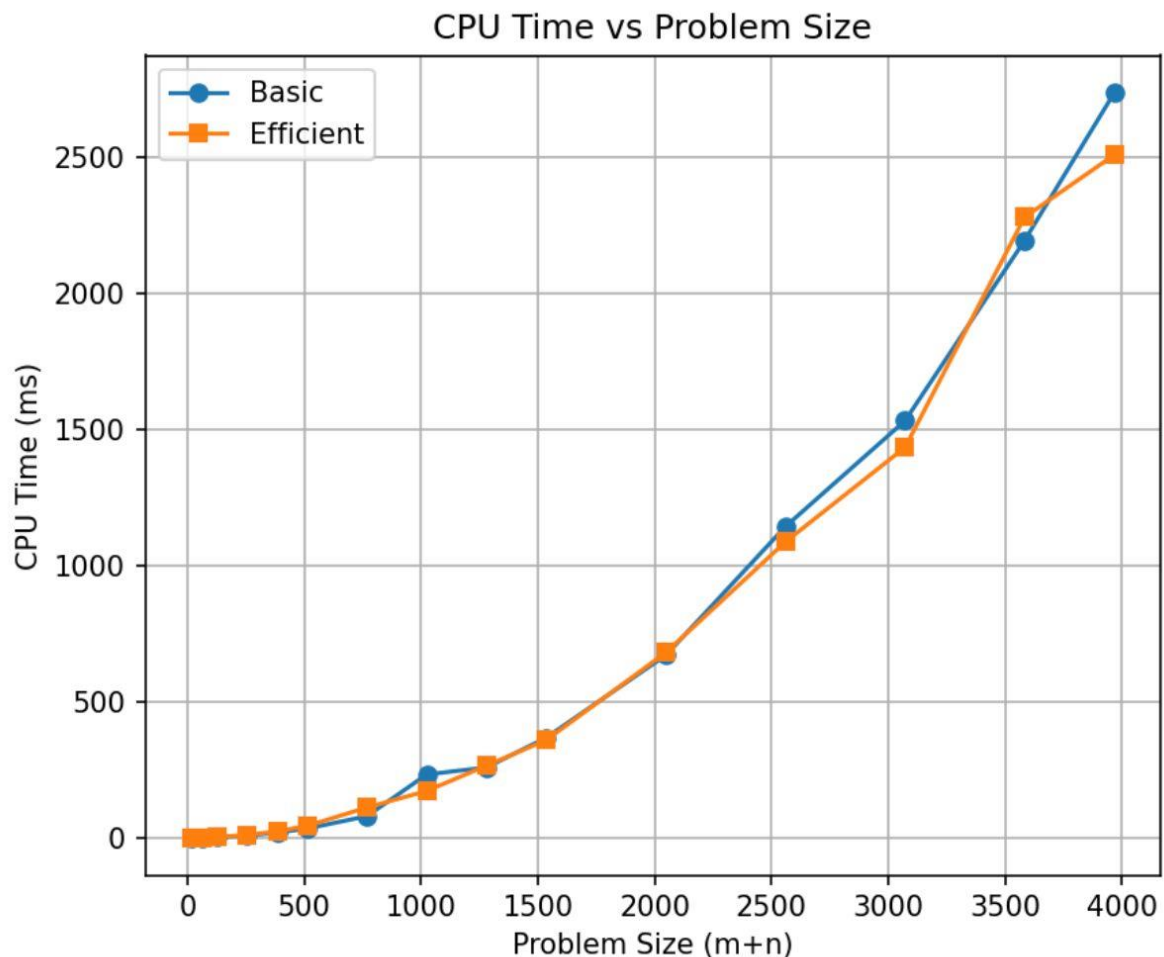
Explanation:

For the basic algorithm, the entire $(M+1) \times (N+1)$ DP table is stored in memory, giving a theoretical space complexity of $\Theta(M \times N)$. This is clearly visible in our graph — the basic curve grows in a quadratic/polynomial fashion. We can verify this using our datapoints: when the problem size doubles from 1024 to 2048, the basic memory grows from 47,120 KB to 156,720 KB, which is roughly 3-4x more, consistent with quadratic growth rather than linear (which would only double).

For the memory-efficient algorithm, we never store the full DP table. Instead, the divide and conquer approach only keeps two rows of the table in memory at any point, reducing space complexity to $\Theta(N)$. This is reflected in our graph — the efficient curve remains nearly flat

across all problem sizes, hovering between 12,000-15,000 KB regardless of input size. For example, memory usage is 32 KB at $m+n=256$ and 432 KB at $m+n=3072$ — barely any difference despite a 12x increase in problem size. The small fluctuations are due to Python runtime overhead rather than actual algorithmic scaling. This near-constant behavior confirms the expected $\Theta(N)$ space complexity of the efficient algorithm.

Graph2 – Time vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

Basic: Polynomial (Quadratic)

Efficient: Polynomial (Quadratic)

Explanation:

The basic algorithm fills a $(M+1) \times (N+1)$ DP table once. There are $\Theta(M \times N)$ cells, and filling each cell does constant time work (calculating 3 candidate costs and taking the minimum). So the theoretical running time is $\Theta(M \times N)$. This matches with our graph: the basic curve starts near 0 ms and increases more than linearly as $M+N$ grows. For example, when the problem size doubles from 1024 to 2048, the time increases from 233 ms to 670 ms (nearly

3-4x more), which is consistent with quadratic growth rather than linear or exponential growth.

The efficient algorithm also evaluates all $(M+1) \times (N+1)$ DP states, but instead of storing the entire table, it uses divide and conquer while keeping only two rows in memory at any time. Since each DP cell still requires constant time work (computing three candidate costs and taking the minimum), the theoretical running time remains $\Theta(M \times N)$. This matches what we see in the graph: the efficient curve also grows polynomially. For example, as the problem size doubles from 1024 to 2048, the time rises from 172 ms to 680 ms, roughly a 4x increase, consistent with quadratic growth rather than linear or exponential behavior. Although the memory usage is significantly lower, the time complexity remains polynomial because the same total number of DP computations must still be performed.

Contribution

(Please mention what each member did if you think everyone in the group does not have an equal contribution, otherwise, write "Equal Contribution")

<USC ID/s>: <Equal Contribution>